Internship Report

Of

AI RAG Chatbot

Submitted

To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

MCA

By

Roll Number of Student: 25SCS2040005354

Name of Student: SHEIKH ANWAR ALI

Batch: 2025-27

CANDIDATE'S DECLARATION

I, SHEIKH ANWAR ALI, do hereby declare that the internship report titled "AI RAG Chatbot" has been completed by me to fulfil the requirement for the award of the degree of MCA. All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own work and has not been submitted to any other institute for any degree/diploma requirement, and I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name: SHEIKH ANWAR ALI

Roll No.: 25SCS2040005354

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout my internship at Elevanware IT Solutions Pvt Ltd. I am thankful for their aspiring guidance, invaluably constructive criticism, and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this project.

I would also like to thank the faculty of the School of Computer Science and Engineering, IILM University, Greater Noida, U.P., for their continuous support and guidance throughout this internship.

I express my gratitude to my parents for their blessings.

Signature

Student Name: SHEIKH ANWAR ALI

Roll No.: 25SCS2040005354

3. Offer Letter

The offer letter issued by the organization, confirming the internship placement, is reproduced below.

ELEVANWARE
IT SOLUTIONS PVT LTD

www.elevanware.com

Wednesday, 1 July 2026

Dear Sheikh Anwar Ali,

Congratulations! We are pleased to offer you the position of **Python Developer Intern** at **Elevanware IT Solutions Pvt Ltd**. We are confident that this internship will provide you with valuable experience in the field of Python development while offering you an opportunity to learn and grow in a professional setting.

During your internship, you will work closely with our experienced team, gaining practical knowledge of Python programming, backend development, databases, APIs, and other relevant technologies. You will be involved in exciting projects that will enhance your skills in developing efficient, scalable, and user-friendly applications using Python.

Thank You

Authorized Signatory

Sincerely,
RIZWAN AHMAD

+91 6388 059 016

3rd floor, plot-no 48, Sector-11, Vikas Nagar,
Lucknow, Ultar Pradesh

hr@elevanware.com

CONTENTS

4. Project Description

4.1 Introduction

This report presents the work carried out during the internship undertaken at Elevanware IT Solutions Pvt Ltd as part of the MCA program at IILM University, Greater Noida, U.P.. The internship focused on building a practical, working application using Python and modern AI tooling, giving hands-on exposure to backend development, API design, and the integration of large language models into real software systems.

The project undertaken was the design and development of an AI-powered chatbot using a Retrieval-Augmented Generation (RAG) approach, built with FastAPI and the Google Gemini API. The chatbot answers user questions by retrieving relevant information from a knowledge base of documents and generating accurate, context-grounded responses, rather than relying purely on a language model's internal knowledge.

4.2 Organization Profile

Elevanware IT Solutions Pvt Ltd is an IT solutions company offering software development and consulting services. The organization works on Python-based backend systems, APIs, and web applications for its clients, and provides internship opportunities for students to gain practical, industry-oriented experience in software development.

Address: 3rd Floor, Plot No. 48, Sector-11, Vikas Nagar, Lucknow, Uttar Pradesh

Website: www.elevanware.com

Email: hr@elevanware.com

4.3 Problem Statement

Organizations and individuals often need quick, accurate answers to questions grounded in a specific set of documents (such as FAQs, policies, or internal knowledge bases), rather than generic answers from a general-purpose chatbot. A plain language model, when asked such questions, may produce answers that are outdated, generic, or

factually incorrect (a problem commonly referred to as hallucination), since it has no direct access to the organization's specific documents. There was a need for a chatbot that could retrieve the most relevant information from a given set of documents and use it to generate accurate, traceable answers.

4.4 Project Objectives

- To design and implement a Retrieval-Augmented Generation (RAG) pipeline that grounds chatbot responses in a specific set of documents.
- To build a REST API backend using FastAPI that exposes the chatbot as a simple, reusable service.
- To implement a document ingestion pipeline capable of processing text and PDF files into a searchable index.
- To integrate a large language model (Google Gemini API) for natural language answer generation.
- To build a simple front-end interface for testing and demonstrating the chatbot.

4.5 Scope of the Project

The scope of the project covers document ingestion, text chunking, retrieval of relevant content based on a user's query, and generation of a grounded answer using a language model. The current implementation supports .txt and .pdf documents as its knowledge source and exposes its functionality through a single REST API endpoint, along with a minimal browser-based chat interface. The project can be extended in the future to support additional file formats, user authentication, conversational memory across multiple turns, and more advanced semantic search using neural embedding models.

4.6 Technologies and Tools Used

The technology stack was selected to keep the application modular and easy to understand. Python provides the core application logic; FastAPI exposes the web service; TF-IDF and cosine similarity support lightweight retrieval; the Google Gemini API provides context-aware language generation; and HTML/JavaScript provides a simple browser interface.

Layer / Area	Technology / Tool	Purpose in the Project
Programming Language	Python 3	Implements ingestion, retrieval, prompt construction, API integration, and application logic.
Web API	FastAPI	Exposes the POST /chat endpoint, validates incoming messages, and returns structured responses.
Server	Uvicorn (ASGI server)	Runs the FastAPI application and handles incoming HTTP requests.
Document Input	pypdf	Extracts source text from PDF files and retains the document name for traceability.
Text Processing	Custom chunking logic	Splits extracted text into overlapping segments to improve retrieval relevance.
Retrieval	scikit-learn (TF-IDF)	Represents chunks and queries as weighted term vectors in a shared vector space.
Ranking	Cosine Similarity	Scores the similarity between the query vector and the indexed document-chunk vectors.
Generation	Google Gemini API	Generates a natural-language answer from the user query and the retrieved context.
Frontend	HTML / JavaScript	Collects user messages, calls the API, and displays answers and source references.
Version Control	Git	Tracks source code changes throughout development.
Development Environment	Visual Studio Code	Supports implementation, debugging, and manual testing.

4.7 System Architecture

The chatbot follows a standard Retrieval-Augmented Generation (RAG) architecture, structured as five clearly separated responsibilities: document ingestion, retrieval, generation, API handling, and presentation. Separation of these responsibilities makes the workflow easier to test, maintain, and extend.

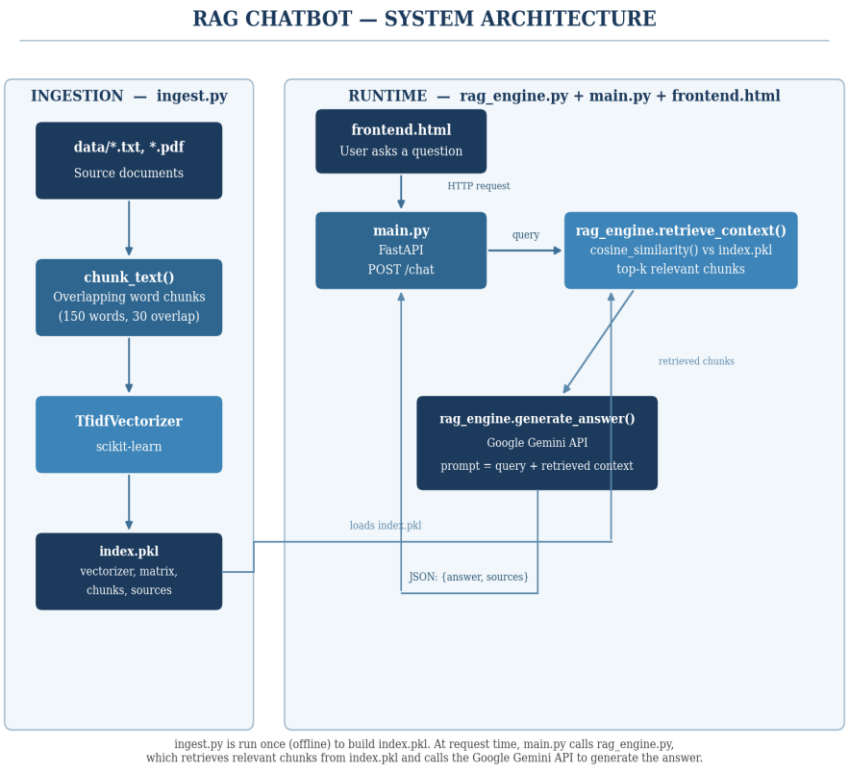

Figure 4.1: Architecture of the implemented RAG chatbot

- Ingestion (ingest.py): Reads .txt/.pdf files from the data/ folder, splits them into overlapping chunks using chunk_text(), builds a TF-IDF representation with scikit-learn's TfidfVectorizer, and saves the vectorizer, matrix, chunks, and source names to index.pkl.
- Retrieval (rag_engine.retrieve_context()): Loads index.pkl, transforms the incoming query into the same TF-IDF vector space, and ranks chunks using cosine_similarity() to select the top-k most relevant chunks.
- Generation (rag_engine.generate_answer()): Builds a prompt from the user's query and the retrieved chunks, sends it to the Google Gemini API, and returns the generated answer along with the source document name(s).
- API (main.py): FastAPI exposes a POST /chat endpoint that accepts a JSON message, calls generate_answer(), and returns {answer, sources} as the response; a GET / health-check endpoint is also provided.

- Presentation (frontend.html): A lightweight HTML/JavaScript chat interface sends the user's message to /chat via fetch() and renders the returned answer in the browser.

Overall data flow: ingest.py builds index.pkl offline, before any chat request. At request time: frontend.html -> main.py (FastAPI) -> rag_engine.retrieve_context() (using index.pkl) -> relevant chunks -> rag_engine.generate_answer() -> Google Gemini API -> generated answer -> JSON response -> frontend.html.

4.8 Methodology

- Requirement analysis: Identified the need for a document-grounded Q&A chatbot and defined the expected inputs/outputs.
- Environment setup: Set up a Python environment with FastAPI, scikit-learn, pypdf, and the Google Gemini SDK.
- Data preparation: Collected sample documents and implemented a chunking strategy with overlap to preserve context across chunk boundaries.
- Index building: Implemented a TF-IDF based vector index for offline, dependency-light document retrieval.
- API development: Built and tested the /chat endpoint, including request validation and error handling.
- Integration: Connected the retrieval layer to the Google Gemini API, using a prompt template that restricts answers to the retrieved context.
- Testing: Verified document ingestion, retrieval accuracy for sample queries, and correct API responses (including edge cases such as empty input).
- Frontend development: Built a minimal chat interface to demonstrate the working system.

4.9 Expected Outcomes

- A working chatbot capable of answering questions grounded in a specific set of documents rather than generic or fabricated information.
- A reusable FastAPI backend that can be integrated into other applications or extended with more advanced retrieval and memory features.

- Practical, hands-on experience with backend API development, information retrieval techniques, and large language model integration.
- A foundation that can be scaled to production using neural embeddings and a dedicated vector database (e.g., ChromaDB or FAISS) in place of the current TF-IDF retriever.

4.10 Certificates of Completion and Other Communication Mail Proof

The internship offer letter issued by the organization, confirming the placement and the terms of the internship, is presented in Section 3 (Offer Letter) of this report and serves as formal proof of the internship undertaken.

5. Bibliography / References

1. FastAPI Documentation. Available at: https://fastapi.tiangolo.com

2. Google. Gemini API Documentation. Available at: https://ai.google.dev/gemini-api/docs

3. scikit-learn Documentation: TfidfVectorizer. Available at: https://scikit-learn.org

4. pypdf Documentation. Available at: https://pypdf.readthedocs.io